

Оптимизация. Локальная и глобальная ОПТИМИЗАЦИЯ

Теория автоматов и формальных языков

Лекция №15

Структура лекции

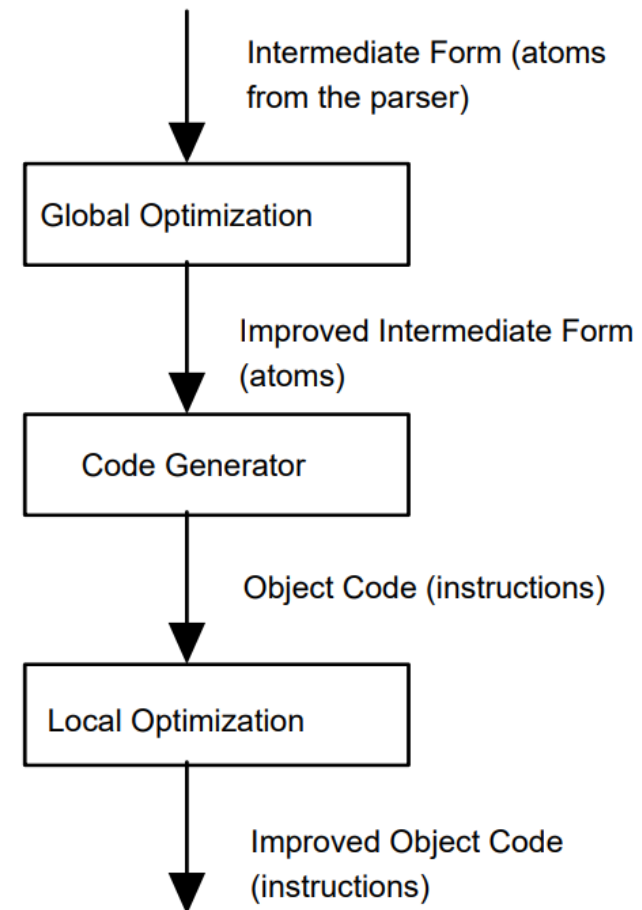
1. Понятие и виды оптимизации кода.
2. Неоднозначности при оптимизации.
3. Направленный ациклический граф (DAG).
4. Виды глобальной оптимизации.
5. Локальная оптимизация.
6. Побочные эффекты.

Понятие оптимизации кода

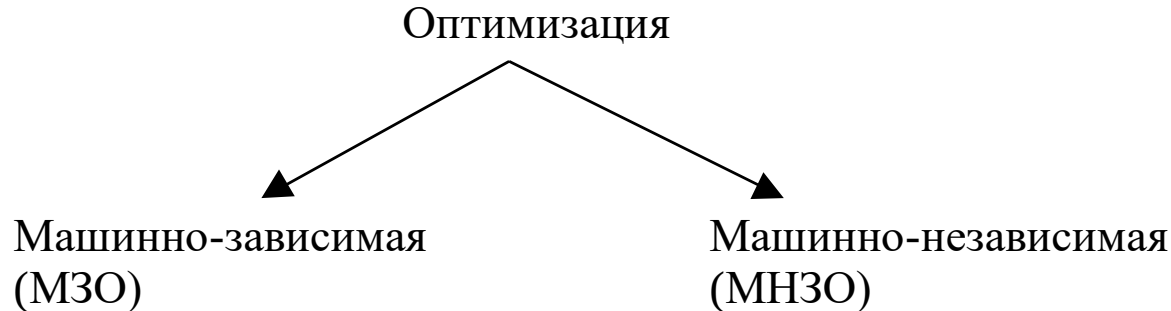
- Оптимизация - это процесс улучшения сгенерированного кода, чтобы уменьшить его потенциальное **время** выполнения и / или уменьшить **пространство**, необходимое для его хранения в памяти.
- Слово «оптимизация», возможно, неверно, поскольку разработанные методы просто пытаются улучшить сгенерированный код, и лишь немногие из них гарантируют создание в любом смысле оптимального (наиболее эффективного из возможных) кода.

Виды оптимизации программ

- Методы **локальной** оптимизации обычно связаны с преобразованием небольших участков кода (включающих всего несколько инструкций) и обычно работают с инструкциями машинного языка, которые генерируются генератором кода.
- С другой стороны, методы **глобальной** оптимизации обычно связаны с более крупными блоками кода или даже с несколькими блоками или модулями и будут применяться к промежуточной форме, структурам атомов или синтаксическим деревьям, выдаваемым синтаксическим анализатором.



Оптимизация программ



- МЗО связана с типом генерируемых команд и включается в фазу генерации кода (т.е. оптимизации подлежат машинные коды).
- МНО – отдельная фаза компилятора, предшествующая генерации кода. Она включается на этапе генерации промежуточного кода – внутренней формы представления программы.

Оптимизация программ

- При решении задачи повышения производительности мы подчеркиваем самое важное свойство компилятора - то, что он **сохраняет семантику** исходной программы.
- Другими словами, цель и поведение объектной программы должны быть точно такими, как указано в исходной программе для всех возможных входных данных.
- Нет мыслимого повышения эффективности, которое может оправдать нарушение этого принципа.

Неоднозначности при КОМПИЛЯЦИИ

- При этом, часто возникают ситуации, когда выражение, указанное в исходной программе, неоднозначно или неясно для конкретной компьютерной архитектуры.
- Например, в выражении $(a + b) * (c + d)$
- компилятор должен будет решить, какое сложение должно быть выполнено первым.

Неоднозначности при КОМПИЛЯЦИИ

- Такие языки, как C, Lisp и APL, в которых есть операторы присваивания, дают еще более интересный пример:

```
a = 2;
```

```
b = ( a * 1 + ( a = 3 ) ) ;
```

- Некоторые разработчики компиляторов считают, что программисты, использующие неоднозначные (**ambiguous**) выражения, подобные этим, заслуживают того, что компилятор может с ними сделать.

Неоднозначности при КОМПИЛЯЦИИ

- Фундаментальный вопрос философии неизбежен при разработке фаз оптимизации.
- Должен ли компилятор вносить значительные изменения и улучшения в исходную программу или он должен уважать решение программиста делать то, что неэффективно или ненужно?

Проблемы оптимизации

- Одна существенная проблема для пользователя компилятора, возникающая на этапах оптимизации, связана с **отладкой**.
- Многие методы оптимизации удаляют ненужный код и перемещают код в объектной программе до такой степени, что это влияет на отладку во время выполнения.
- Для решения этой проблемы в большинстве современных и доступных компиляторов есть переключатель, с помощью которого можно включать или выключать оптимизацию.
- Другое решение этой проблемы, использованное IBM в начале 1970-х годов для своего компилятора PL/1, - создание двух отдельных компиляторов.
 - The checkout compiler,
 - The optimizing compiler.

Что получится в результате?

```
{  
int a;  
(a = 2) + (a = 3);  
cout << a;  
}
```

Почему указанные выражения нельзя считать эквивалентными?

- $a = f(x) + f(x) + f(x) ;$
- $a = 3 * f(x) ;$

Глобальная оптимизация

- Методы глобальной оптимизации обычно принимают в качестве входных данных промежуточную форму в виде последовательности атомов (трехадресный код) или синтаксических деревьев.
- Мы более подробно рассмотрим оптимизацию общих подвыражений в базовых блоках, а затем кратко рассмотрим некоторые другие методы глобальной оптимизации.

Оптимизация выражений

- Последовательность атомов, выводимая парсером, явно не является оптимальной, в ней есть много ненужных и избыточных атомов.
- Например, при анализе оператора C ++:

$a = (b + c) * (b + c) ;$

- Может быть сгенерирован такой набор атомов:

```
(ADD, b, c, T1)
(ADD, b, c, T2)
(MUL, T1, T2, T3)
(MOV, T3, , a)
```

Figure 7.2 Atom Sequence for $a = (b + c) * (b + c) ;$

Оптимизация выражений

```
(ADD, b, c, T1)
(ADD, b, c, T2)
(MUL, T1, T2, T3)
(MOV, T3, , a)
```

Figure 7.2 Atom Sequence for $a = (b + c) * (b + c) ;$

```
(ADD, b, c, T1)
(MUL, T1, T1, a)
```

Figure 7.3 Optimized Atom Sequence for $a = (b + c) * (b + c) ;$

Оптимизация выражений

- Оптимизация затруднена или невозможна при наличии атомов меток и прыжков:

```
(ADD, b, c, T1)
(LBL, L1)
(ADD, b, c, T2)
(MUL, T1, T2, T3)
(TST, b, c,, 1, L3)
(MOV, T3,, a)
```

Figure 7.4 Example of an Atom Sequence Which Cannot be Optimized

Базовые блоки

- Методы оптимизации, которые мы продемонстрируем, могут быть реализованы только в определенных подпоследовательностях: это строки атомов, которые мы называем базовыми блоками.
- **Базовый блок** - это часть атомов, которая не содержит атомов меток или ветвей (то есть LBL, TST, JMP). На рис. 7.5 мы показываем, что последовательность атомов на рис. 7.4 разделена на три основных блока.

(ADD, b, c, T1)	Block 1
(LBL, L1)	
(ADD, b, c, T2)	Block 2
(MUL, T1, T2, T3)	
(TST, b, c, , 1, L3)	
(MOV, T3, , a)	Block 3

Figure 7.5 Basic Blocks Contain No LBL, TST, or JMP Atoms

Directed Acyclic Graph

- Мы используем **направленный ациклический граф** «**Directed Acyclic Graph**», или DAG, для реализации такого подхода оптимизации.

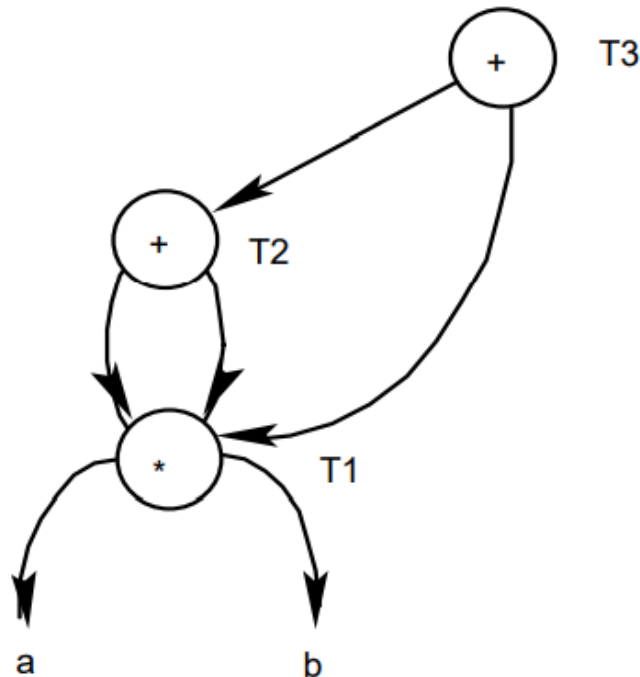


Figure 7.6 Example of a DAG

Построение DAG

- Мы начнем с создания групп DAG для простых арифметических выражений.
- Схемы DAG также можно использовать для оптимизации полных операторов присваивания и блоков операторов.
- Чтобы построить DAG, учитывая последовательность атомов, представляющую арифметическое выражение с бинарными операциями, используется следующий алгоритм:

Построение DAG

1. Прочитать атом.
2. Если операция и операнды совпадают с частью существующей группы DAG (т. е. если они образуют дочернюю группу DAG), добавить метку результата в список меток родительского элемента и повторить действия, начиная с шага 1.
 - В противном случае выделить новый узел для каждого операнда, которого еще нет в DAG, и узел для операции. Обозначить узел операции именем результата операции.
3. Соединить рабочий узел с двумя операндами направленными дугами, чтобы было ясно, какой операнд левый, а какой правый.
4. Повторить с шага 1.

Построение DAG

- В качестве примера построим DAG для выражения

$$a * b + a * b + a * b.$$

- У этого выражения явно есть некоторые общие подвыражения, которые должны сделать его пригодным для оптимизации.
- Последовательность атомов, выводимая парсером, будет следующей:

(MUL, a, b, T1)

(MUL, a, b, T2)

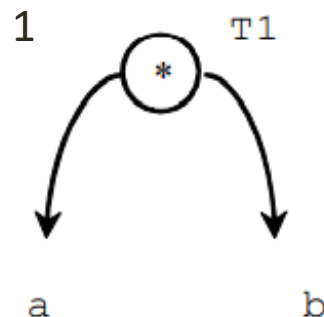
(ADD, T1, T2, T3)

(MUL, a, b, T4)

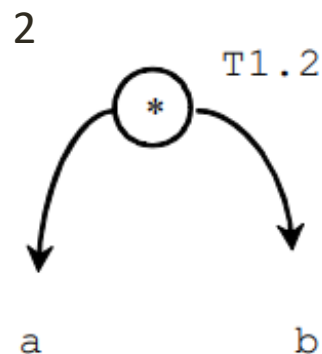
(ADD, T3, T4, T5)

Построение DAG

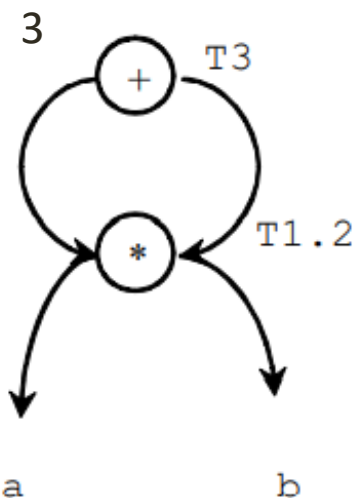
- DAG - это графическое представление вычислений, необходимых для построения исходного выражения, в котором мы определили общие подвыражения.



(MUL, a, b, T1)



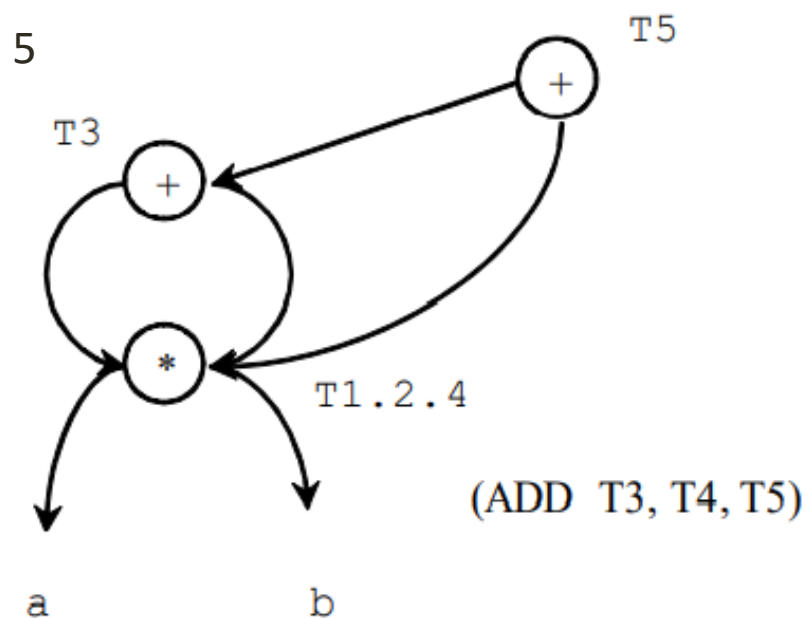
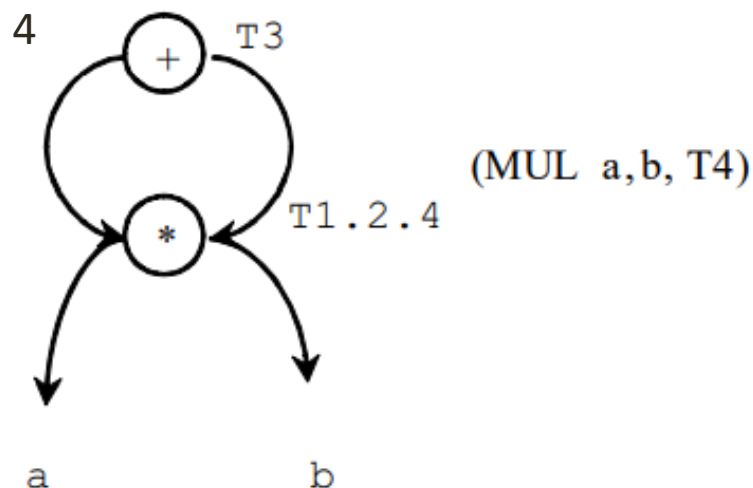
(MUL, a, b, T2)



(ADD, T1, T2, T3)

Построение DAG

- Выражение $a * b$ встречается трижды в исходном выражении $a * b + a * b + a * b$.
- Три атома, соответствующие этим подвыражениям, сохраняют результаты в T1, T2 и T4.
- Поскольку вычисление необходимо выполнить только один раз, эти три атома объединяются в один узел в DAG с меткой T1.2.4.
- После этого любой атом, который использует T1, T2 или T4 в качестве операнда, будет указывать на T1.2.4.

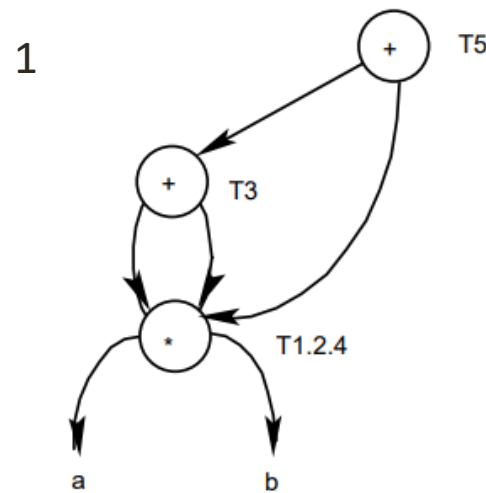


Построение набора атомов по DAG

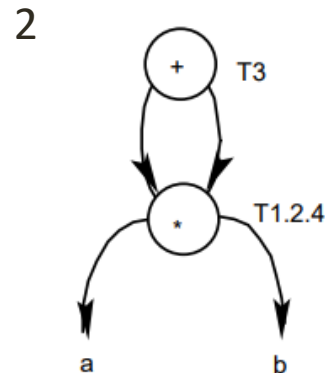
- Теперь мы готовы преобразовать DAG в базовый блок атомов.
- Приведенный ниже алгоритм будет генерировать атомы (в обратном порядке), в которых все общие подвыражения оцениваются только один раз:
 1. Выбрать любой узел, не имеющий входящих дуг (изначально должен быть только один такой узел, представляющий значение всего выражения).
 2. Вывести атом для операций и операндов.
 3. Удалить этот узел и его исходящие дуги из группы DAG.
 4. Повторить действия, начиная с шага 1, пока в группе DAG все еще остаются рабочие узлы.

Построение набора атомов по DAG

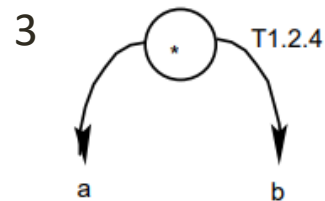
- Составной узел, такой как T1.2.4, обозначается его полным именем, а не просто T1 или T2 по соглашению, что помогает проверять наличие ошибок.
- Мы начали с цепочки из пяти атомов и улучшили ее до эквивалентной цепочки из трех атомов.
- Это приведет к значительной экономии времени выполнения и пространства, необходимого для объектной программы.



(ADD, T3, T1.2.4, T5)



(ADD, T1.2.4, T1.2.4, T3)



(MUL, a,b, T1.2.4)

Обработка унарных и других операций

- С помощью этого метода можно легко обрабатывать унарные операции.
- Поскольку унарная операция имеет только один операнд, ее узел будет иметь только одну дугу, указывающую на операнд, но во всех других отношениях алгоритмы, данные для построения групп DAG и генерации оптимизированных последовательностей атомов, остаются неизменными.
- Следовательно, этот метод хорошо обрабатывает выражения, включающие операции с любым количеством операндов, хотя для наших целей операции обычно имеют два операнда.

Другие виды глобальной оптимизации

- Теперь мы рассмотрим несколько других распространенных методов глобальной оптимизации, однако мы не будем вдаваться в их реализацию.
1. Недостижимый код.
 2. Обнаружение инвариантов цикла.
 3. Оптимизация математических выражений.
 4. Алгебраические преобразования.

Недостижимый код

- Недостижимый код - это атом или последовательность атомов, которые не могут быть выполнены, потому что поток управления не может достичь этой последовательности атомов.

(JMP, L1)		
(MUL, a, b, T1)		
(SUB, T1, c, T2)	⇒	(JMP, L1)
(ADD, T2, d, T3)		(LBL, L2)
(LBL, L2)		

Анализ потока данных

- **Анализ потока данных (Data Flow Analysis)** - это формальный способ отслеживания того, как информация об элементах данных перемещается по программе, и используется для многих методов оптимизации.
- Хотя анализ потока данных выходит за рамки лекции, мы рассмотрим некоторые виды оптимизации, которые могут возникнуть в результате такого анализа.
- Одним из таких методов оптимизации является **устранение мертвого кода**, который включает определение того, действительно ли используются вычисления, указанные в исходной программе, и влияют ли они на вывод программы.

Анализ потока данных

- Устранение мертвого кода:

```
{  
    a = b + c * d;      //This statement has no effect and can be removed  
    b = c * d / e;  
    c = b - 3;  
    a = b - c;  
    cout << a << b << c ;  
}
```

Обнаружение инвариантов цикла

- Другой метод оптимизации, использующий анализ потока данных, - это **обнаружение инвариантов цикла**.
- Инвариант цикла - это код внутри цикла, который имеет дело со значениями данных, которые остаются постоянными при повторении цикла.

```
{
    for (i=0; i<1000; i++)
        { a = sqrt (x);           // loop invariant
          vector[i] = i * a;
        }
}

{
    a = sqrt (x);           // loop invariant
    for (i=0; i<1000; i++)
        {
            vector[i] = i * a;
        }
}
```

Оптимизация математических преобразований

- Остальные методы глобальной оптимизации, которые будут рассмотрены в этом разделе, включают математические преобразования.
- Их использование не рекомендуется повсеместно, и что часто, используя их, разработчик компилятора выполняет преобразования, которые нежелательны для программиста исходного кода.
- Например, здесь решающим является вопрос о значении арифметического переполнения.

Сворачивание констант

- **Сворачивание констант** (Constant Folding) - это процесс обнаружения операций над константами, которые могут выполняться во время компиляции, а не во время выполнения.
- Если эти вычисления происходят в небольшом цикле, сворачивание констант может привести к значительному улучшению времени выполнения (за счет небольшого времени компиляции).
- Такой вид оптимизации иногда называют «Вычисления на этапе компиляции».

```
{  
    a = 2 * 3;           // a must be 6  
    b = c + a * a;       // a * a must be 36  
}  
  
{  
    a = 6;  
    b = c + 36;  
}
```

Figure 7.11 Constant Folding

Уменьшение силы

- Еще одно математическое преобразование называется **уменьшением силы** (Reduction in strength).
- Эта оптимизация является результатом того факта, что некоторые операции требуют больше времени, чем другие, практически на всех архитектурах.
- Например, можно ожидать, что умножение займет значительно больше времени, чем сложение. Таким образом, умножение $2 * x$ всегда будет выполняться медленнее, чем сложение $x + x$.
- Аналогично, если есть оператор возведения в степень, $x ^ 2$ наверняка будет медленнее, чем $x * x$.

Алгебраические преобразования

- Наш последний пример математических преобразований включает в себя **алгебраические преобразования** с использованием таких свойств, как коммутативность, ассоциативность и свойство распределения, которые кратко изложены ниже.
- Мы не считаем, что эти свойства обязательно верны при работе с компьютерной арифметикой из-за конечной точности числовых данных. Тем не менее, они используются во многих компиляторах, поэтому мы кратко их обсудим здесь.

$$a + b == b + a$$

Addition is commutative

$$(a + b) + c == a + (b + c)$$

Addition is associative

$$a * (b + c) == a * b + a * c$$

Multiplication distributes over addition

Figure 7.12 Algebraic Identities

Алгебраические преобразования

- Ранее рассмотренные методы оптимизации не обнаружили бы общие подвыражения в выражениях:

$$a = b + c;$$

$$b = c + d + b;$$

- Лишняя операция умножения в выражении:

$$a * c + b * c$$

- может быть устранена за счет свойства дистрибутивности умножения:

$$(a + b) * c$$

Исключение общих подвыражений

- **Оптимизация линейных участков** – это процедура, позволяющая не рассчитывать одно и то же выражение несколько раз, исключая общие подвыражения.
1. представить выражение в форме, пригодной для обнаружения общих подвыражений;
 2. определить эквивалентность двух и более подвыражений;
 3. исключить повторяющиеся;
 4. изменить команды так, чтобы учесть это исключение.

Исключение общих подвыражений

пусть выражение $A = c*d*(d*c+b)$ записано в виде тетрад:

- Упорядочим операнды в алфавитном порядке (там, где это возможно) [1];
- определим границы операторов и найдем общие подвыражения (это (1) и (2)) [2];
- затем исключим подвыражение (2).
- После чего заменим далее везде T2 на T1 [3].

```
1. * c d T1
2. * d c T2
3. + T2 b T3
4. * T1 T3 T4
5. = A T4
```

1

```
1. * c d T1
2. * c d T2
3. + T2 b T3
4. * T1 T3 T4
5. = A T4
```

2

```
1. * c d T1
2. + T1 b T3
3. * T1 T3 T4
4. = A T4
```

3

Алгебраические преобразования

- Создатели компиляторов, использующие эти методы, создают более эффективные программы для большого числа программистов, которые хотят и ценят улучшения, но рискуют сгенерировать нежелательный код для небольшого числа программистов, которым требуется, чтобы алгебраические выражения вычислялись точно так, как указано в исходной программе.

Пример глобальной оптимизации

```
(LBL, L1)
(TST, a, b, , 1, L2)
(SUB, a, 1, a)
(MUL, x, 2, b)
(ADD, x, y, z)
(ADD, 2, 3, z)
(JMP, L1)
(SUB, a, b, a)
(MUL, x, 2, z)
(LBL, L2)
```

Reduction in strength
Elimination of dead code
Constant folding, loop invariant

Unreachable code
Unreachable code

```
(MOV, 5, , z)
(LBL, L1)
(TST, a, b, , 1, L2)
(SUB, a, 1, a)
(ADD, x, x, b)
(JMP, L1)
(LBL, L2)
```

Локальная оптимизация

- В этом разделе мы обсуждаем методы локальной оптимизации.
- Определение локальных и глобальных методов значительно различается в учебниках по проектированию компиляторов.
- Мы считаем, что любая **оптимизация, применяемая к сгенерированному коду, считается локальной.**
- Методы локальной оптимизации часто называют оптимизацией с помощью глазка, поскольку они обычно включают преобразования инструкций, которые расположены близко друг к другу в объектной программе.

Локальная оптимизация

- Здесь будут обсуждаться три типа методов локальной оптимизации:
 - оптимизация чтения / записи (Load/Store optimization)
 - оптимизация перехода через переход (Jump over jump opt.)
 - простая алгебраическая оптимизация (Simple algebraic opt.)
-
- Кроме того, схемы распределения регистров, подобные рассмотренной ранее, можно рассматривать как локальную оптимизацию, хотя обычно они выполняются в самом генераторе кода.

Локальная оптимизация

- Парсер выполнит перевод выражения $a + b - c$ в следующий поток атомов:

(ADD, a, b, T1)

(SUB, T1, c, T2)

- Простейшая схема генератора кода будет генерировать три инструкции, соответствующие каждому атому:
 - загрузить первый операнд в регистр (LOD),
 - выполнить операцию
 - и сохранить результат обратно в память (STO).
- Генератор кода затем выдаст из атомов следующие инструкции:

```
LOD    R1, a
ADD    R1, b
STO    R1, T1
LOD    R1, T1
SUB    R1, c
STO    R1, T2
```

Оптимизация чтения/записи

load/store optimization

- Обратите внимание, что третья и четвертая инструкции в этой последовательности совершенно не нужны, поскольку сохраняемое и загружаемое значение уже находится в месте назначения.
- Вышеупомянутая последовательность из шести инструкций может быть оптимизирована до следующей последовательности из четырех инструкций путем исключения промежуточных инструкций Load и Store, как показано ниже:

```
LOD    R1, a
ADD     R1, b
STO     R1, T1
LOD     R1, T1
SUB     R1, c
STO     R1, T2
```



```
LOD    R1, a
ADD     R1, b
SUB     R1, c
STO     R1, T2
```

Jump over jump optimization

- Другой метод локальной оптимизации, который мы называем **оптимизацией перехода через переход**, очень распространен и связан с ненужными переходами.
- Большой эффективности можно добиться, переписав условную логику.
- Хороший пример этого можно найти в компиляторе C ++ для оператора

```
if (a > b) a = b ;
```

Это может быть переведено в следующий поток атомов:

```
(TST, a, b, , 3, L1)  
(JMP, L2)  
(LBL, L1)  
(MOV, b, , a)  
(LBL, L2)
```

Jump over jump optimization

- Может быть сгенерирован указанный ниже набор инструкций:

```
(TST, a, b,, 3, L1)
(JMP, L2)
(LBL, L1)
(MOV, b,, a)
(LBL, L2)
```

```
    LOD    R1,a
    CMP    R1,b,3           //Is R1 > b?
    JMP    L1
    CMP    0,0,0           // Unconditional Jump
    JMP    L2
```

L1:

```
    LOD    R1,b
    STO    R1,a
```

L2:

Jump over jump optimization

- Нет необходимости реализовывать эту логику с двумя инструкциями Jump.

```

                                LOD    R1,a
                                CMP    R1,b,3           //Is R1 > b?
                                JMP     L1
                                CMP    0,0,0           // Unconditional Jump
                                JMP     L2
L1:
                                LOD     R1,b
                                STO     R1,a
L2:
```

- Мы можем значительно улучшить этот код, проверив, является ли условие ложным, а не истинным, как показано ниже:

```

                                LOD     R1,a
                                CMP     R1,b,4           // Is R1 <= b?
                                JMP     L1
                                LOD     R1,b
                                STO     R1,a
L1:
```

Jump over jump optimization

- Эта оптимизация могла произойти в промежуточной форме (то есть мы могли бы рассматривать ее как глобальную оптимизацию), но такой вид скачка через скачок может происходить по ряду других причин.
- Например, в некоторых архитектурах условный переход - это «короткий» переход (к ограниченному диапазону адресов), а безусловный переход - «длинный» переход.
- Таким образом, до тех пор, пока не будет сгенерирован код, неизвестно, находится ли цель условного перехода в пределах досягаемости или необходим безусловный переход, чтобы прыгнуть так далеко.

Оптимизация булевых выражений

- Метод основан на использовании свойств булевых выражений.

Вместо

if (a and b and c) then <операторы> endif

надо сгенерировать команды таким образом, чтобы исключались лишние проверки.

if not a then goto Label

if not b then goto Label

if not c then goto Label

 <операторы>

Label: //метка перехода

Простые алгебраические оптимизации

- Последний пример методов локальной оптимизации включает простые алгебраические преобразования, которые зависят от машины и называются **простыми алгебраическими оптимизациями**.

- Например, можно исключить следующие инструкции:

MUL R1, 1

ADD R1, 0

- потому что умножение значения на 1 или прибавление 0 к значению не должно изменять это значение.
- Кроме того, инструкцию (MUL R1, 0) можно улучшить, заменив ее на (CLR R1), потому что результат будет всегда равняться 0 (это фактически уменьшение трансформации силы).

Проблемы, связанные с оптимизацией

- необходимо сопоставлять ожидаемый выигрыш от повышения эффективности объектной программы с дополнительными накладными расходами, связанными с **увеличением времени компиляции, надежности и сложности самого компилятора.**
- С оптимизаций зачастую связаны такие проблемы, как даже ухудшение кода, **сложность трассировки и отладки оптимизированных программ.**
- Побочные эффекты.

Итоги

Оптимизация связана с улучшением машинного кода и / или промежуточного кода, генерируемого другими фазами компилятора.

Эти улучшения могут привести к сокращению времени выполнения и / или места для объектной программы.

Существует два основных класса оптимизации: глобальная и локальная.

Глобальная оптимизация работает с атомами или синтаксическими деревьями, создаваемыми интерфейсом компилятора, а локальная оптимизация работает с инструкциями, выдаваемыми генератором кода.

Термин «оптимизация» используется для этой фазы компилятора, хотя никогда нельзя быть уверенным в том, что будет произведен оптимальный код ни в пространстве, ни во времени.

Писатель компилятора должен быть осторожен, чтобы не изменить цель программы при применении методов оптимизации. Многие из этих методов могут сильно повлиять на инструменты отладки; следовательно, отладка обычно выполняется на неоптимизированном коде.

Итоги

Глобальная оптимизация применяется к блокам кода в промежуточной форме (атомам), которые не содержат атомов меток или ветвей. Они называются базовыми блоками и могут быть представлены **ориентированными ациклическими графами** (DAG), в которых каждый внутренний узел представляет собой операцию со ссылками на его операнды.

Основные методы глобальной оптимизации:

1. **недостижимый код** - код, который никогда не может быть выполнен и, следовательно, может быть удален;
2. **мертвый код** - код, который может быть выполнен, но не может повлиять на вывод программы и, следовательно, может быть удален;
3. **инвариантный код цикла** - код, который находится внутри цикла, но который на самом деле не обязательно должен быть в цикле и может быть перемещен из цикла;
4. **сворачивание констант** - обнаружение арифметических операций над константами, которые могут быть вычислены во время компиляции, а не во время выполнения;
5. **снижение силы** - замена медленных арифметических операций на быстрые;
6. **алгебраические преобразования** - преобразования, включающие коммутативные, ассоциативные и дистрибутивные свойства арифметики.

Итоги

Типы локальной оптимизации:

- 1. оптимизация загрузки/сохранения** - устранение ненужных инструкций загрузки и сохранения в архитектуре загрузки / сохранения;
 - 2. оптимизация перехода через переход** - замена двух инструкций перехода одним переходом путем инвертирования логики;
 - 3. простая алгебраическая оптимизация** - исключение добавления или вычитание 0, умножение или деление на 1.
- Эти методы оптимизации не являются обязательными, но они используются в большинстве современных компиляторов из-за значительных улучшений объектной программы.